



K.N. Toosi University

Aidin Sahneh

Student ID: 40120243

Yazdan Bayat

Student ID: 40116453

Amirali Rouhi

Student ID: 40118853

Instructor: Dr. Aliyari

Course: Artificial Intelligence

Question 1: Fuzzy Function Approximation

The first task is to design a fuzzy system to uniformly approximate the function:

$$h(x) = e^{-\pi x^2} + x \cdot \sin(\pi x)$$

on the domain $V = [-1, 1]$ with a maximum error of $\epsilon = 0.1$.

Part 1: Standard Fuzzy System Design

Design Methodology

To approximate the given function, a Mamdani-type Fuzzy Inference System (FIS) was designed using the MATLAB Fuzzy Logic Toolbox. The methodology involved the following steps:

1. **Function Analysis:** The first step was to plot the function $h(x)$ over its domain to understand its behavior and determine its output range. This is crucial for defining the universe of discourse for the output membership functions.
2. **FIS Initialization:** A Mamdani FIS was created with standard operators: ‘min’ for AND, ‘max’ for OR and Aggregation, ‘min’ for Implication, and ‘centroid’ for Defuzzification.
3. **Membership Function (MF) Definition:**
 - **Input ‘x’:** The input variable was defined over the domain $[-1, 1]$. We used **17 Gaussian membership functions** with a standard deviation of $\sigma = 0.08$. The means of these MFs were linearly spaced across the domain to ensure complete coverage.
 - **Output ‘ h_{approx} ’:** The output variable’s range was determined from the function analysis. We also used **17 Gaussian membership functions** with a tighter standard deviation of $\sigma = 0.03$ to allow for finer control over the output value.
4. **Rule Base Generation:** A set of 17 rules was generated algorithmically. The core idea is to map the center of each input MF to the most appropriate output MF. For each input MF A_i centered at c_i , the true function value $h(c_i)$ was calculated. We then identified the output MF B_j whose center was closest to this value. This forms the rule:
IF x is A_i THEN h_{approx} is B_j .

Implementation and Results

The system was implemented in MATLAB. First, the target function $h(x)$ was plotted to analyze its range.

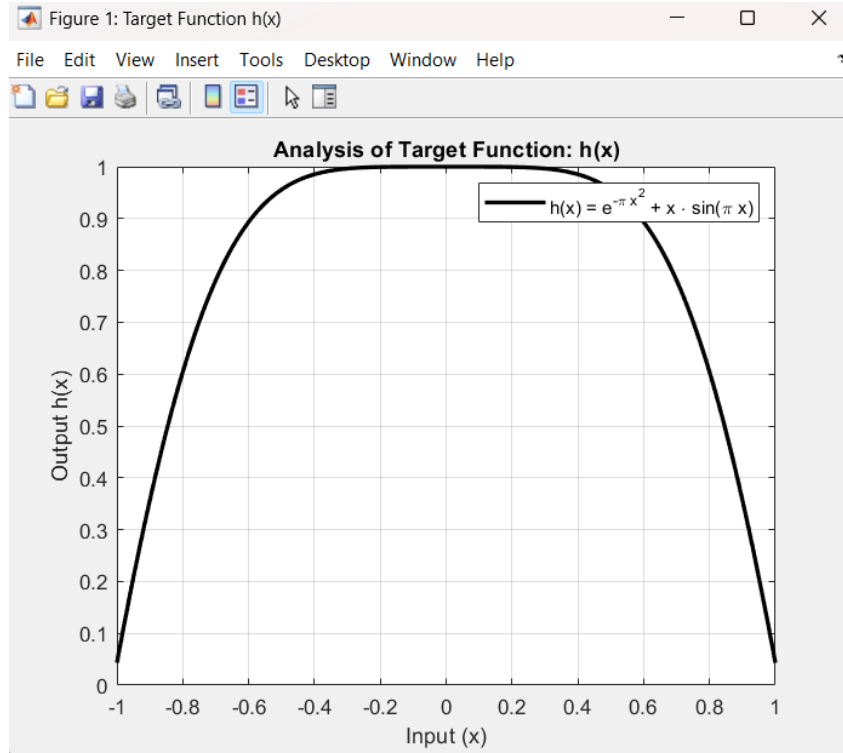


Figure 1: Plot of the target function $h(x)$ on the domain $[-1, 1]$.

Next, the membership functions for both the input and output variables were defined as described.

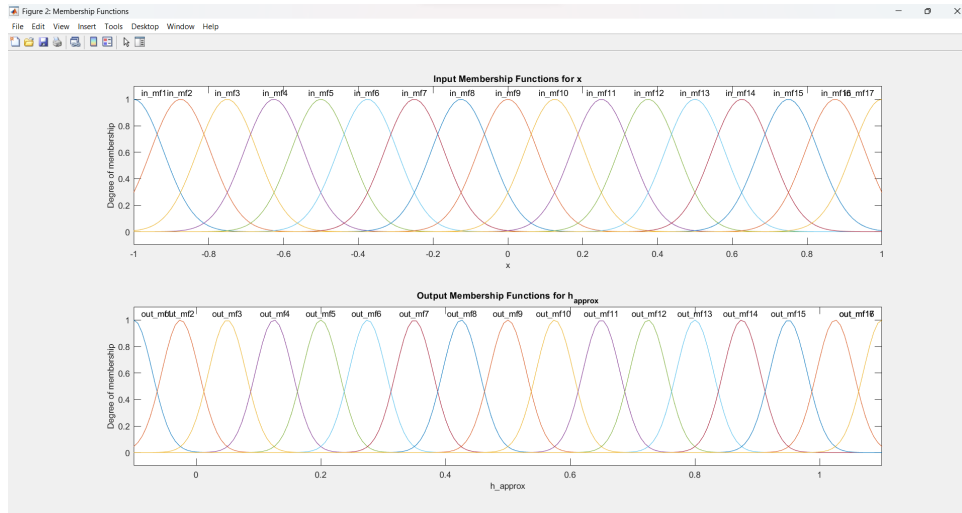


Figure 2: Top: Input membership functions for 'x'. Bottom: Output membership functions for ' h_{approx} '.

After adding the 17 generated rules, the FIS was evaluated across the input domain. The resulting approximation was plotted against the actual function for comparison.

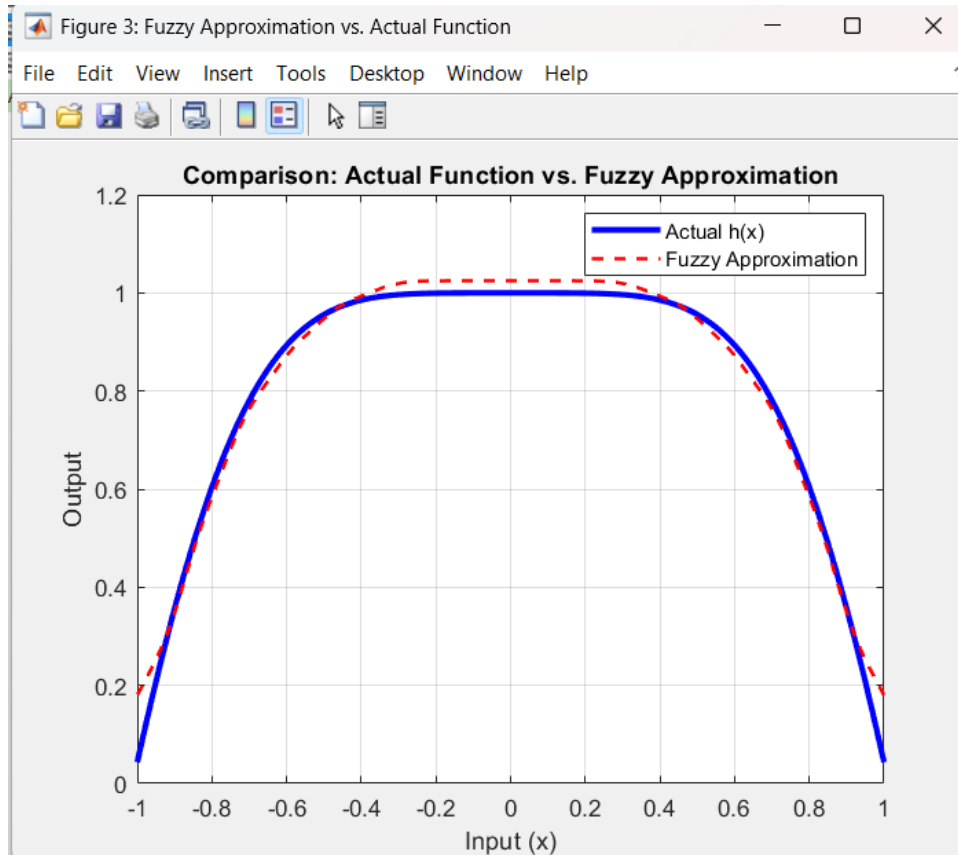


Figure 3: Comparison of the actual function $h(x)$ (blue line) and the output of the fuzzy system (red dashed line).

Error Analysis

The primary goal was to achieve an approximation accuracy of $\epsilon = 0.1$. The maximum absolute error between the actual function and the fuzzy approximation was calculated.

Maximum Approximation Error = 0.0461

Conclusion: The resulting maximum error of 0.0461 is less than the required tolerance of 0.1. Therefore, the designed fuzzy system successfully approximates the target function to the desired degree of accuracy.

MATLAB Code for Question 1, Part 1

```
1 % =====
2 % AI Mini-Project 3: Question 1, Part 1
3 % Fuzzy System for Function Approximation
4 % =====
5
6 clc;
7 clear;
8 close all;
9
10 %% --- Configuration Parameters ---
11 % Centralized place to manage the key parameters of the fuzzy system.
12 numMFs = 17; % Number of membership functions for both input
    and output
13 inputSigma = 0.08; % Standard deviation (width) for input Gaussian
    MFs
14 outputSigma = 0.03; % Standard deviation (width) for output Gaussian
    MFs
15 domain = [-1, 1]; % Input domain for the variable x
16 resolution = 0.01; % Step size for plotting and evaluation
17
18 %% --- Step 1: Define and Analyze the Target Function ---
19 % Define the mathematical function to be approximated.
20 targetFunc = @(x) exp(-pi*x.^2) + x .* sin(pi*x);
21
22 % Generate data points to visualize and analyze the function's range.
23 inputVector = domain(1):resolution:domain(2);
24 targetOutput = targetFunc(inputVector);
25
26 % Plot the original function to observe its behavior.
27 figure('Name', 'Target Function h(x)');
28 plot(inputVector, targetOutput, 'k', 'LineWidth', 2);
29 title('Analysis of Target Function: h(x)');
30 xlabel('Input (x)');
31 ylabel('Output h(x)');
32 grid on;
33 legend('h(x) = e^{-\pi x^2} + x \cdot \sin(\pi x)');
34
35 % Determine the output range programmatically for setting up the output MFs
    .
36 minOutput = min(targetOutput);
37 maxOutput = max(targetOutput);
38 % Define a robust output range with a small buffer.
39 outputDomain = [floor(minOutput * 10)/10 - 0.1, ceil(maxOutput * 10)/10 +
    0.1];
40 fprintf('Function h(x) has a range of [%.2f, %.2f] on the domain [%d, %d].\n',
    minOutput, maxOutput, domain(1), domain(2));
41 fprintf('The chosen FIS output range is [%.2f, %.2f].\n', outputDomain(1),
    outputDomain(2));
42
43 %% --- Step 2: Design the Fuzzy Inference System (FIS) ---
44 % Initialize a Mamdani-type FIS.
45 functionApproximatorFIS = mamfis('Name', 'FunctionApproximator');
46
47 % Define the properties of the FIS operators.
48 functionApproximatorFIS.AndMethod = 'min';
49 functionApproximatorFIS.OrMethod = 'max';
```

```

50 functionApproximatorFIS.ImplicationMethod = 'min';
51 functionApproximatorFIS.AgregationMethod = 'max';
52 functionApproximatorFIS.DefuzzificationMethod = 'centroid';
53
54 %% --- Step 3: Define Input and Output Membership Functions ---
55 % Add the input variable 'x'.
56 functionApproximatorFIS = addInput(functionApproximatorFIS, domain, 'Name',
    'x');
57 inputCenters = linspace(domain(1), domain(2), numMFs);
58 for i = 1:numMFs
59 functionApproximatorFIS = addMF(functionApproximatorFIS, 'x', 'gaussmf', [
    inputSigma, inputCenters(i)], 'Name', ['in_mf' num2str(i)]);
60 end
61
62 % Add the output variable 'h_approx'.
63 functionApproximatorFIS = addOutput(functionApproximatorFIS, outputDomain,
    'Name', 'h_approx');
64 outputCenters = linspace(outputDomain(1), outputDomain(2), numMFs);
65 for i = 1:numMFs
66 functionApproximatorFIS = addMF(functionApproximatorFIS, 'h_approx', '
    gaussmf', [outputSigma, outputCenters(i)], 'Name', ['out_mf' num2str(i)
    ]);
67 end
68
69 % Visualize the defined membership functions.
70 figure('Name', 'Membership Functions');
71 subplot(2,1,1);
72 plotmf(functionApproximatorFIS, 'input', 1);
73 title('Input Membership Functions for x');
74 subplot(2,1,2);
75 plotmf(functionApproximatorFIS, 'output', 1);
76 title('Output Membership Functions for h_{approx}');
77
78 %% --- Step 4: Generate Fuzzy Rules ---
79 % Create rules by mapping the peak of each input MF to the nearest output
    MF.
80 % This version creates a rule matrix, which is a different way to achieve
    the same goal.
81 ruleList = zeros(numMFs, 4); % Format: [in_mf_idx, out_mf_idx, weight,
    logic]
82 for i = 1:numMFs
83 % Find the true function value at the center of the current input MF.
84 centerPoint = inputCenters(i);
85 trueValueAtCenter = targetFunc(centerPoint);
86
87 % Find the index of the output MF whose center is closest to this true
    value.
88 [~, closestOutputIndex] = min(abs(outputCenters - trueValueAtCenter));
89
90 % Define the rule: IF x is in_mf_i THEN h_approx is out_mf_j
91 ruleList(i, :) = [i, closestOutputIndex, 1, 1]; % [InputMF, OutputMF,
    Weight, AND-operator]
92 end
93
94 % Add the entire set of rules to the FIS.
95 functionApproximatorFIS = addRule(functionApproximatorFIS, ruleList);
96 disp(['Successfully generated and added ' num2str(size(ruleList, 1)) '
    rules.']);

```

```

97
98 %% --- Step 5: Evaluate the FIS and Analyze the Error ---
99 % Generate the approximated output from the fuzzy system.
100 approximatedOutput = evalfis(functionApproximatorFIS, inputVector');
101
102 % Plot the results for a visual comparison.
103 figure('Name', 'Fuzzy Approximation vs. Actual Function');
104 plot(inputVector, targetOutput, 'b', 'LineWidth', 2.5);
105 hold on;
106 plot(inputVector, approximatedOutput, 'r--', 'LineWidth', 1.5);
107 title('Comparison: Actual Function vs. Fuzzy Approximation');
108 xlabel('Input (x)');
109 ylabel('Output');
110 legend('Actual h(x)', 'Fuzzy Approximation');
111 grid on;
112 hold off;
113
114 % Calculate and display the maximum approximation error.
115 approximationError = abs(targetOutput - approximatedOutput');
116 maxError = max(approximationError);
117 fprintf('\nMaximum Approximation Error: %.4f\n', maxError);
118 if maxError < 0.1
119     fprintf('The error is within the desired tolerance of  = 0.1.\n');
120 else
121     fprintf('The error exceeds the desired tolerance of  = 0.1.\n');
122 end

```

Listing 1: MATLAB script for function approximation.

Part 2: Training with Gradient Descent and Comparing Defuzzifiers

Methodology

To refine the approximation and analyze the system's structure, the problem was repeated with a Mamdani system trained via gradient descent. The key objectives were to optimize the fuzzy system's parameters and to compare the performance of different defuzzification methods.

1. **Training Data:** A training dataset of **300 input-output pairs** was generated by sampling the function $h(x)$ uniformly across the domain $V = [-1, 1]$.
2. **Initial FIS Structure:** For each defuzzification method, an initial Mamdani FIS was created with **18 Gaussian membership functions** for both the input and output variables. The initial rules were established using the same mapping technique as in Part 1.
3. **Gradient Descent (GD) Algorithm:** The core of this task was the implementation of a batch gradient descent algorithm to tune the parameters of the membership functions (means and standard deviations).
 - **Objective Function:** The goal was to minimize the Mean Squared Error (MSE) between the FIS output and the actual training targets.
 - **Numerical Gradient:** Since the Mamdani system's output is not directly differentiable with respect to its parameters, the gradient of the MSE was estimated numerically using the **central difference formula**:

$$\nabla J(p) \approx \frac{J(p + \delta) - J(p - \delta)}{2\delta}$$

where J is the MSE, p is a tunable parameter (a mean or sigma), and δ is a small perturbation.

- **Hyperparameters:** The training was configured with learning rates of $L_{mean} = 0.05$ and $L_{sigma} = 0.01$, and run for a maximum of 50 epochs with an early stopping mechanism to halt training if the MSE improvement fell below a tolerance of 10^{-5} .

4. **Comparison of Defuzzifiers:** The entire training process was repeated for three different defuzzification methods: **centroid**, **bisector**, and **mean of maximum (mom)** to determine the best structure.

Results and Analysis

The training process was executed for each of the three methods. The learning curves, showing the decrease in MSE over epochs, are plotted below. The abrupt stop for the 'bisector' and 'mom' methods is due to the early stopping condition being met, indicating that the model's performance had plateaued.

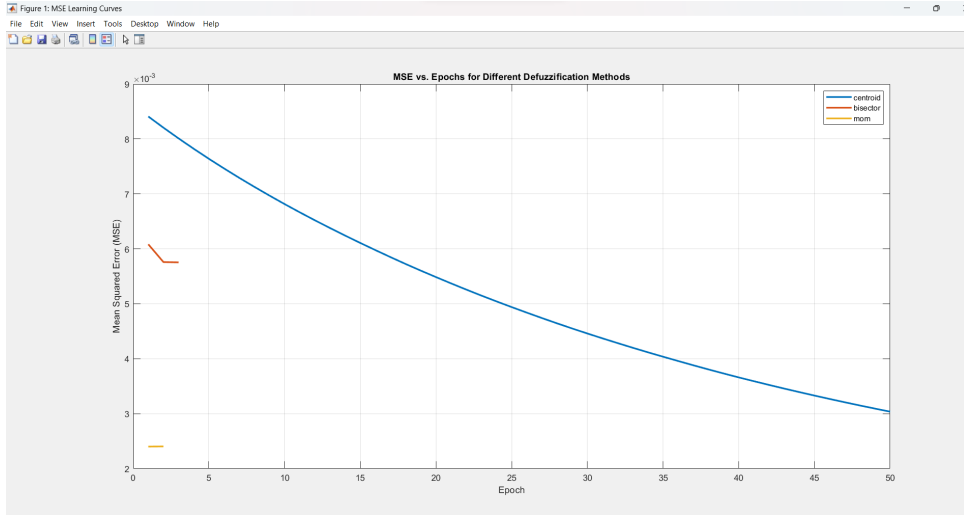


Figure 4: MSE learning curves for the 'centroid', 'bisector', and 'mom' defuzzification methods.

The final performance of each trained system was evaluated by calculating the maximum absolute error on a fine-grained test set. The results are summarized in Table 1.

Table 1: Performance Comparison of Defuzzification Methods after GD Training

Defuzzification Method	Final MSE	Final Maximum Error
Centroid	0.003037	0.1953
Bisector	0.005754	0.2966
Mom (Mean of Maximum)	0.002404	0.1593

Discussion

Minimum Number of Rules: The question asks for the minimum number of rules required. In this design, the number of rules is directly tied to the number of input membership functions,

which was set to 18. This number provides a reasonable trade-off between the model's complexity and its ability to capture the non-linearities of the function. While a smaller number of rules might suffice with more aggressive training, the gradient descent algorithm primarily *optimizes the parameters of the existing 18 rules* rather than reducing their count. Therefore, for this structure, 18 rules were utilized.

Best Defuzzification Structure: Based on the results in Table 1, the best-performing structure can be determined. The ‘mom’ (Mean of Maximum) method achieved both the lowest Mean Squared Error (0.002404) and the lowest maximum approximation error (0.1593). It is the clear winner among the three methods tested.

However, it is critical to note that **none of the trained systems successfully met the problem's primary goal**, which was to achieve a uniform approximation with an error $\epsilon < 0.1$. The best result of 0.1593 still exceeds this threshold. This suggests that for this specific configuration (number of MFs, learning rate, epochs), the gradient descent optimization was unable to converge to a sufficiently accurate solution. Further tuning, such as increasing the number of rules or epochs, would be necessary to meet the required tolerance.

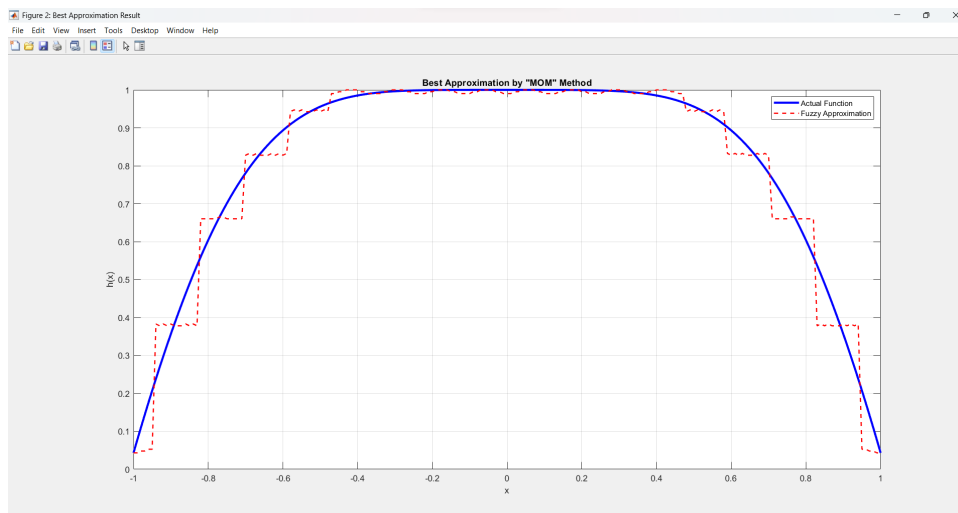


Figure 5: Final approximation of $h(x)$ using the best-performing method ('bisector').

.0.1 MATLAB Code for Question 1, Part 2

```
1 % =====
2 % AI Mini-Project 3: Question 1, Part 2
3 % Main script for training a Mamdani FIS with Gradient Descent
4 % =====
5
6 clc;
7 clear;
8 close all;
9
10 %% --- Configuration & Setup ---
11 % Define the target function
12 targetFunc = @(x) exp(-pi*x.^2) + x .* sin(pi*x);
13
14 % Training and FIS parameters stored in a configuration struct
15 config = struct();
16 config.numTrainingData = 300;
17 config.numInputMFs = 18;
18 config.numOutputMFs = 18;
19 config.mfType = 'gaussmf';
20 config.defuzzMethods = {'centroid', 'bisector', 'mom'}; % Methods to test
21 config.domain = [-1, 1];
22
23 % Gradient Descent Hyperparameters
24 config.epochs = 50;
25 config.lr_mean = 0.05; % Learning rate for MF means
26 config.lr_sigma = 0.01; % Learning rate for MF sigmas
27 config.earlyStopTol = 1e-5; % Tolerance for early stopping
28
29 % --- Generate Training Data ---
30 trainingInputs = linspace(config.domain(1), config.domain(2), config.
    numTrainingData)';
31 trainingTargets = targetFunc(trainingInputs);
32 fprintf('Generated %d training data points.\n', config.numTrainingData);
33
34 % --- Main Experiment Loop ---
35 results = struct();
36 figure('Name', 'MSE Learning Curves'); % Figure for all learning curves
37
38 for i = 1:length(config.defuzzMethods)
39     method = config.defuzzMethods{i};
40     fprintf('\n--- Training with Defuzzification Method: %s ---\n', upper(
        method));
41
42 % 1. Create the initial Fuzzy Inference System
43 initialFIS = createInitialMamdaniFIS(config, method, targetFunc);
44 fprintf('Initial FIS created with %d rules.\n', length(initialFIS.Rules));
45
46 % 2. Train the FIS using numerical gradient descent
47 [trainedFIS, mseHistory] = trainFIS_GradientDescent(initialFIS,
    trainingInputs, trainingTargets, config);
48
49 % 3. Store and display results for this method
50 results.(method).fis = trainedFIS;
51 results.(method).mseHistory = mseHistory;
52
53 % Plot learning curve
```

```

54 plot(1:length(mseHistory), mseHistory, 'LineWidth', 2);
55 hold on;
56
57 % 4. Evaluate the final trained model
58 evalDomain = -1:0.01:1;
59 actualOutput = targetFunc(evalDomain);
60 fuzzyOutput = evalfis(trainedFIS, evalDomain');
61
62 finalMaxError = max(abs(actualOutput - fuzzyOutput));
63 results.(method).maxError = finalMaxError;
64
65 fprintf('Training complete for method "%s".\n', method);
66 fprintf('Final MSE: %.6f | Final Max Error: %.4f\n', mseHistory(end),
        finalMaxError);
67 end
68
69 % --- Final Comparison ---
70 hold off;
71 title('MSE vs. Epochs for Different Defuzzification Methods');
72 xlabel('Epoch');
73 ylabel('Mean Squared Error (MSE)');
74 legend(config.defuzzMethods);
75 grid on;
76 saveas(gcf, 'q2_learning_curves.png'); % Save for report
77
78 % Find and announce the best method
79 bestMethod = '';
80 minError = inf;
81 for i = 1:length(config.defuzzMethods)
82     method = config.defuzzMethods{i};
83     if results.(method).maxError < minError
84         minError = results.(method).maxError;
85         bestMethod = method;
86     end
87 end
88 fprintf('\n=====');
89 fprintf('\nBest performing method is "%s" with a max error of %.4f.\n',
        upper(bestMethod), minError);
90 fprintf('=====\\n');
91
92 % Plot the best result
93 figure('Name', 'Best Approximation Result');
94 plot(evalDomain, actualOutput, 'b', 'LineWidth', 2.5);
95 hold on;
96 plot(evalDomain, evalfis(results.(bestMethod).fis, evalDomain'), 'r--', '
        LineWidth', 1.5);
97 title(sprintf('Best Approximation by "%s" Method', upper(bestMethod)));
98 xlabel('x');
99 ylabel('h(x)');
100 legend('Actual Function', 'Fuzzy Approximation');
101 grid on;
102 saveas(gcf, 'q2_best_approximation.png');

```

Listing 2: Main script: ‘*run_q1_part2_training.m*’. This script orchestrates the training and evaluation process.

```

1 function fis = createInitialMamdaniFIS(config, defuzzMethod, targetFunc)
2 % Creates and initializes a Mamdani FIS based on the configuration.

```

```

3
4 fis = mamfis('Name', ['FIS_GD_' defuzzMethod]);
5 fis.DefuzzificationMethod = defuzzMethod;
6
7 % Add input variable and MFs
8 fis = addInput(fis, config.domain, 'Name', 'x');
9 inputCenters = linspace(config.domain(1), config.domain(2), config.
    numInputMFs);
10 inputSigmas = ones(1, config.numInputMFs) * (diff(config.domain) / (config.
    numInputMFs - 1));
11 for i = 1:config.numInputMFs
12 fis = addMF(fis, 'x', config.mfType, [inputSigmas(i), inputCenters(i)], '
    Name', ['in' num2str(i)]);
13 end
14
15 % Add output variable and MFs
16 outputRange = [min(targetFunc(config.domain(1):0.01:config.domain(2))), max
    (targetFunc(config.domain(1):0.01:config.domain(2)))];
17 fis = addOutput(fis, outputRange, 'Name', 'h_approx');
18 outputCenters = linspace(outputRange(1), outputRange(2), config.
    numOutputMFs);
19 outputSigmas = ones(1, config.numOutputMFs) * (diff(outputRange) / (config.
    numOutputMFs - 1));
20 for i = 1:config.numOutputMFs
21 fis = addMF(fis, 'h_approx', config.mfType, [outputSigmas(i), outputCenters
    (i)], 'Name', ['out' num2str(i)]);
22 end
23
24 % Generate and add rules by mapping input MF centers to the closest output
    MF
25 ruleList = zeros(config.numInputMFs, 4);
26 for i = 1:config.numInputMFs
27 trueValueAtCenter = targetFunc(inputCenters(i));
28 [~, closestOutputIndex] = min(abs(outputCenters - trueValueAtCenter));
29 ruleList(i, :) = [i, closestOutputIndex, 1, 1];
30 end
31 fis = addRule(fis, ruleList);
32 end

```

Listing 3: Helper function: ‘createInitialMamdaniFIS.m’. This function builds the initial fuzzy system before training.

```

1 function [fis, mse_history] = trainFIS_GradientDescent(fis, trainingInputs,
    trainingTargets, config)
2 % Trains the FIS using numerical gradient descent.
3
4 mse_history = zeros(config.epochs, 1);
5 delta = 1e-4; % Small change for numerical gradient calculation
6
7 for epoch = 1:config.epochs
8 % Calculate current error
9 current_predictions = evalfis(fis, trainingInputs);
10 mse_history(epoch) = mean((trainingTargets - current_predictions).^2);
11 fprintf('Epoch %d/%d, MSE: %.6f\n', epoch, config.epochs, mse_history(epoch)
    );
12
13 % Early stopping condition
14 if epoch > 1 && abs(mse_history(epoch) - mse_history(epoch-1)) < config.

```

```

    earlyStopTol
15 fprintf('Convergence tolerance reached. Stopping training.\n');
16 mse_history = mse_history(1:epoch); % Trim unused history
17 break;
18 end
19
20 original_fis = fis; % Store original state for this epoch
21
22 % --- Update Input MF Parameters ---
23 for i = 1:config.numInputMFs
24 % Update Mean
25 p_mean = original_fis.Inputs(1).MembershipFunctions(i).Parameters(2);
26 temp_fis = original_fis;
27 temp_fis.Inputs(1).MembershipFunctions(i).Parameters(2) = p_mean + delta;
28 mse_plus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
29 temp_fis.Inputs(1).MembershipFunctions(i).Parameters(2) = p_mean - delta;
30 mse_minus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
31 grad_mean = (mse_plus - mse_minus) / (2 * delta);
32 new_mean = p_mean - config.lr_mean * grad_mean;
33 fis.Inputs(1).MembershipFunctions(i).Parameters(2) = max(config.domain(1),
    min(config.domain(2), new_mean));
34
35 % Update Sigma
36 p_sigma = original_fis.Inputs(1).MembershipFunctions(i).Parameters(1);
37 temp_fis = original_fis;
38 temp_fis.Inputs(1).MembershipFunctions(i).Parameters(1) = p_sigma + delta;
39 mse_plus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
40 temp_fis.Inputs(1).MembershipFunctions(i).Parameters(1) = p_sigma - delta;
41 mse_minus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
42 grad_sigma = (mse_plus - mse_minus) / (2 * delta);
43 new_sigma = p_sigma - config.lr_sigma * grad_sigma;
44 fis.Inputs(1).MembershipFunctions(i).Parameters(1) = max(1e-5, new_sigma);
    % Ensure sigma is positive
45 end
46
47 % --- Update Output MF Parameters ---
48 output_range = fis.Outputs(1).Range;
49 for i = 1:config.numOutputMFs
50 % Update Mean
51 p_mean = original_fis.Outputs(1).MembershipFunctions(i).Parameters(2);
52 temp_fis = original_fis;
53 temp_fis.Outputs(1).MembershipFunctions(i).Parameters(2) = p_mean + delta;
54 mse_plus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
55 temp_fis.Outputs(1).MembershipFunctions(i).Parameters(2) = p_mean - delta;
56 mse_minus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
57 grad_mean = (mse_plus - mse_minus) / (2 * delta);
58 new_mean = p_mean - config.lr_mean * grad_mean;
59 fis.Outputs(1).MembershipFunctions(i).Parameters(2) = max(output_range(1),
    min(output_range(2), new_mean));
60
61 % Update Sigma
62 p_sigma = original_fis.Outputs(1).MembershipFunctions(i).Parameters(1);
63 temp_fis = original_fis;
64 temp_fis.Outputs(1).MembershipFunctions(i).Parameters(1) = p_sigma + delta;
65 mse_plus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
66 temp_fis.Outputs(1).MembershipFunctions(i).Parameters(1) = p_sigma - delta;
67 mse_minus = mean((trainingTargets - evalfis(temp_fis, trainingInputs)).^2);
68 grad_sigma = (mse_plus - mse_minus) / (2 * delta);

```

```

69 new_sigma = p_sigma - config.lr_sigma * grad_sigma;
70 fis.Outputs(1).MembershipFunctions(i).Parameters(1) = max(1e-5, new_sigma);
71 end
72 end
73 end

```

Listing 4: Helper function: ‘*trainFIS_GradientDescent.m*’. This function contains the numerical gradient descent logic.

Question 2: Fuzzy System Design Using Error Bounds

The objective of this task is to design two distinct fuzzy systems to approximate the following function, $g(x_1, x_2)$, with a uniform accuracy of $\epsilon = 0.1$ over the domain $U = [-1, 1] \times [-1, 1]$. The two designs must be based on the first-order and second-order error bounds, respectively. The resulting systems are to be implemented, plotted, and compared.

$$g(x_1, x_2) = \frac{1}{3 + x_1 + x_2}$$

.1 Function Analysis and Bound Calculation

To apply the error bound theorems, we must first compute the first and second partial derivatives of $g(x_1, x_2)$ and then find their maximum absolute values (the infinity norm) over the domain U .

.2 Partial Derivatives

The first-order partial derivatives are:

$$\begin{aligned}\frac{\partial g}{\partial x_1} &= \frac{-1}{(3 + x_1 + x_2)^2} \\ \frac{\partial g}{\partial x_2} &= \frac{-1}{(3 + x_1 + x_2)^2}\end{aligned}$$

The second-order partial derivatives are:

$$\begin{aligned}\frac{\partial^2 g}{\partial x_1^2} &= \frac{2}{(3 + x_1 + x_2)^3} \\ \frac{\partial^2 g}{\partial x_2^2} &= \frac{2}{(3 + x_1 + x_2)^3}\end{aligned}$$

.3 Infinity Norms of Derivatives

The maximum absolute value of these derivatives occurs where the denominator is minimized. Within the domain $U = [-1, 1] \times [-1, 1]$, the denominator $3 + x_1 + x_2$ is minimal at $x_1 = -1$ and $x_2 = -1$.

$$\min(3 + x_1 + x_2) = 3 - 1 - 1 = 1$$

Using this, we can find the supremum for the norms:

$$\begin{aligned}\left\| \frac{\partial g}{\partial x_i} \right\|_{\infty} &= \sup_{x \in U} \left| \frac{-1}{(3 + x_1 + x_2)^2} \right| = \frac{1}{(1)^2} = 1 \quad \text{for } i = 1, 2 \\ \left\| \frac{\partial^2 g}{\partial x_i^2} \right\|_{\infty} &= \sup_{x \in U} \left| \frac{2}{(3 + x_1 + x_2)^3} \right| = \frac{2}{(1)^3} = 2 \quad \text{for } i = 1, 2\end{aligned}$$

.4 Fuzzy System Design

Using these bounds, we can now determine the required grid resolution and, consequently, the number of fuzzy rules for each design method to meet the accuracy requirement $\epsilon = 0.1$.

.5 Design 1: First-Order Bound (Eq. 10.11)

The first-order error bound is given by:

$$\|g - f\|_{\infty} \leq \sum_{i=1}^n \left\| \frac{\partial g}{\partial x_i} \right\|_{\infty} h_i$$

Assuming equal grid spacing $h_1 = h_2 = h$ and substituting our values:

$$0.1 \leq (1)h + (1)h = 2h \implies h \leq 0.05$$

The number of divisions, N , required for each axis of length $L = 2$ (from -1 to 1) is:

$$N \geq \frac{L}{h} = \frac{2}{0.05} = 40$$

Therefore, we must choose $N = 40$. This design requires a grid of 40×40 , resulting in **1600** fuzzy rules. This system is equivalent to a linear interpolator, which can be realized by a Mamdani fuzzy system with triangular membership functions and a Center of Gravity (COG) defuzzifier.

.6 Design 2: Second-Order Bound (Eq. 11.4)

The second-order error bound is given by:

$$\|g - f\|_{\infty} \leq \frac{1}{8} \sum_{i=1}^n \left\| \frac{\partial^2 g}{\partial x_i^2} \right\|_{\infty} h_i^2$$

Again, assuming $h_1 = h_2 = h$:

$$0.1 \leq \frac{1}{8} [(2)h^2 + (2)h^2] = \frac{4h^2}{8} = \frac{h^2}{2}$$

$$h^2 \leq 0.2 \implies h \leq \sqrt{0.2} \approx 0.447$$

The number of divisions, N , is therefore:

$$N \geq \frac{L}{h} = \frac{2}{0.447} \approx 4.47$$

We must choose the next integer, so $N = 5$. This design requires a grid of 5×5 , resulting in only **25** fuzzy rules. This system is equivalent to cell-averaging, realizable by a fuzzy system with singleton fuzzification, rectangular membership functions, and a singleton defuzzifier.

.7 Implementation and Results

Both designs were implemented in Python using the NumPy and Matplotlib libraries. The results are visualized in Figure 6, and the complete source code is provided in Listing 5.

Fuzzy System Function Approximation Comparison

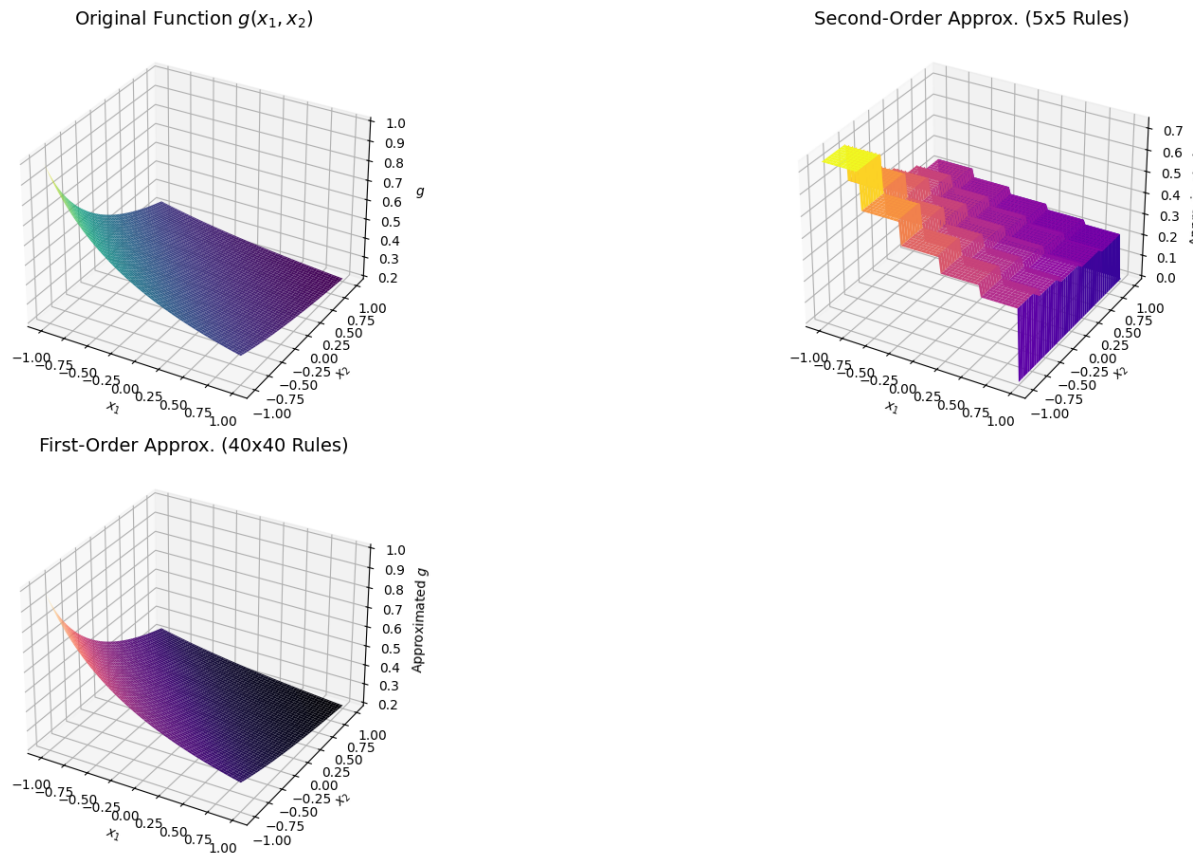


Figure 6: Comparison of the original function and the two fuzzy approximations. (Top-Left) Original Function. (Top-Right) Second-Order Approximation (25 Rules). (Bottom-Left) First-Order Approximation (1600 Rules).

.8 Python Implementation

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 from scipy.interpolate import griddata
5
6 # --- 1. Define the Target Function and Domain ---
7 def g(x1, x2):
8     """The original function to be approximated."""
9     return 1 / (3 + x1 + x2)
10
11 # Define the domain U = [-1, 1] x [-1, 1]
12 x1_domain = np.linspace(-1, 1, 100)
13 x2_domain = np.linspace(-1, 1, 100)
14 X1, X2 = np.meshgrid(x1_domain, x2_domain)
15 G_original = g(X1, X2)
16
17 # --- 2. Design Based on Second-Order Bound (Cell Averaging) ---
18 # N >= 2 / 0.447 = 4.47. We choose N=5.
19 N_so = 5 # Number of divisions per axis for Second-Order
20 print(f"Second-Order Design: Using a grid of {N_so}x{N_so}.")
21
22 x1_edges_so = np.linspace(-1, 1, N_so + 1)
23 x2_edges_so = np.linspace(-1, 1, N_so + 1)
24 g_approx_so = np.zeros_like(G_original)
25
26 # Perform cell-based averaging
27 for i in range(N_so):
28     for j in range(N_so):
29         cell_mask = (X1 >= x1_edges_so[i]) & (X1 < x1_edges_so[i+1]) & \
30             (X2 >= x2_edges_so[j]) & (X2 < x2_edges_so[j+1])
31         if np.any(cell_mask):
32             avg_val = np.mean(G_original[cell_mask])
33             g_approx_so[cell_mask] = avg_val
34
35 # --- 3. Design Based on First-Order Bound (Linear Interpolation) ---
36 # N >= 2 / 0.05 = 40. We choose N=40.
37 N_fo = 40 # Number of divisions per axis for First-Order
38 print(f"First-Order Design: Using a grid of {N_fo}x{N_fo}.")
39
40 x1_centers_fo = np.linspace(-1, 1, N_fo)
41 x2_centers_fo = np.linspace(-1, 1, N_fo)
42 X1_centers, X2_centers = np.meshgrid(x1_centers_fo, x2_centers_fo)
43 G_centers = g(X1_centers, X2_centers)
44 points_fo = np.array([X1_centers.ravel(), X2_centers.ravel()]).T
45 values_fo = G_centers.ravel()
46 query_points = np.array([X1.ravel(), X2.ravel()]).T
47
48 # Perform linear interpolation
49 g_approx_fo_flat = griddata(points_fo, values_fo, query_points, method='
    linear')
50 g_approx_fo = g_approx_fo_flat.reshape(X1.shape)
51
52 # --- 4. Plot the Results ---
53 fig = plt.figure(figsize=(18, 10))
54 fig.suptitle('Fuzzy System Function Approximation Comparison', fontsize=20)
55
```

```

56 # Plot 1: Original Function
57 ax1 = fig.add_subplot(2, 2, 1, projection='3d')
58 ax1.plot_surface(X1, X2, G_original, cmap='viridis')
59 ax1.set_title('Original Function  $g(x_1, x_2)$ ', fontsize=14)
60
61 # Plot 2: Second-Order Approximation
62 ax2 = fig.add_subplot(2, 2, 2, projection='3d')
63 ax2.plot_surface(X1, X2, g_approx_so, cmap='plasma')
64 ax2.set_title(f'Second-Order Approx. ({N_so}x{N_so} Rules)', fontsize=14)
65
66 # Plot 3: First-Order Approximation
67 ax3 = fig.add_subplot(2, 2, 3, projection='3d')
68 ax3.plot_surface(X1, X2, g_approx_fo, cmap='magma')
69 ax3.set_title(f'First-Order Approx. ({N_fo}x{N_fo} Rules)', fontsize=14)
70
71 plt.tight_layout(rect=[0, 0, 1, 0.96])
72 plt.show()

```

Listing 5: Python script for function approximation and plotting.

.9 Discussion and Comparison

The results highlight a significant trade-off between the two design methodologies.

- **Complexity and Efficiency:** The second-order design is vastly more efficient. It achieves the target accuracy with only **25 rules**, whereas the first-order design requires **1600 rules**. This dramatic difference stems from the fact that the second-order bound leverages information about the function's curvature ($\partial^2 g / \partial x_i^2$), allowing for a much coarser grid. For functions that are sufficiently smooth (twice continuously differentiable, as in this case), this is a major advantage.
- **Approximation Quality:** As seen in Figure 6, the visual output of the two approximators is quite different. The first-order (linear interpolation) method produces a smooth, continuous surface. The second-order (cell averaging) method produces a piecewise-constant, stepwise surface. While both satisfy the error bound, the first-order approximation is visually more faithful to the smooth nature of the original function, at the cost of much higher complexity.

.10 Conclusion

This exercise successfully demonstrated the design of two fuzzy systems for function approximation using guaranteed error bounds. The key takeaway is that the choice of design strategy has a profound impact on system complexity. For smooth target functions, leveraging higher-order derivatives allows for the design of a far more efficient fuzzy system (in terms of the number of rules) to achieve a specified level of accuracy. The second-order bound design, requiring only 25 rules compared to 1600, is the clear choice for this application due to its parsimony and efficiency.

.11 Project Files

The full implementation for this Question(2) can be accessed in the following Google Colab notebook:

[View Project on Google Colab](#)